Enludus Shorts Publisher — complete upload script (common/upload.py)

Full, working source as used in production. Public repository: https://github.com/enludus-daily/yt-pipeline/blob/main/common/upload.py

Scope: https://www.googleapis.com/auth/youtube.upload only. Endpoint: videos.insert (part=snippet,status). Token stored locally in .cache/ (git-ignored).

```
"""Upload one video to our own YouTube channel (YouTube Data API v3, videos.insert).

usage:
  python common/upload.py <video.mp4> <meta.json> [--dry-run]
  python common/upload.py --auth [--no-browser]      # sign in once and save the token

meta.json (see examples/gojuon-14-ketchup.json):
  title, description, tags[], categoryId ("20" = Gaming), privacyStatus (private/unlisted/public),
  publishAt (optional, RFC3339; needs privacyStatus "private"), madeForKids (bool),
  containsSyntheticMedia (bool: realistic altered/synthetic content), defaultLanguage, defaultAudioLanguage

Until the API project passes YouTube's audit, every upload is locked to private by YouTube,
whatever privacyStatus says. Upload by hand in YouTube Studio until then.

Files (never commit them; .cache/ is git-ignored):
  .cache/client_secret.json   OAuth client (Desktop app) downloaded from Google Cloud Console
  .cache/youtube-token.json   saved token (refreshed automatically)
"""
import argparse
import json
import os
import sys
import time

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
CLIENT = os.environ.get("YT_CLIENT_SECRET", os.path.join(ROOT, ".cache", "client_secret.json"))
TOKEN = os.environ.get("YT_TOKEN", os.path.join(ROOT, ".cache", "youtube-token.json"))
LOG = os.path.join(ROOT, "output", "uploads.jsonl")
SCOPES = ["https://www.googleapis.com/auth/youtube.upload"]


def credentials(no_browser=False, force=False):
    from google.auth.transport.requests import Request
    from google.oauth2.credentials import Credentials
    from google_auth_oauthlib.flow import InstalledAppFlow

    creds = None
    if os.path.exists(TOKEN) and not force:
        creds = Credentials.from_authorized_user_file(TOKEN, SCOPES)
    if creds and creds.valid:
        return creds
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    else:
        if not os.path.exists(CLIENT):
            sys.exit(f"OAuth client not found: {CLIENT} (download it from Google Cloud Console > Credentials)")
        flow = InstalledAppFlow.from_client_secrets_file(CLIENT, SCOPES)
        if no_browser:
            # For a machine without a browser: open the URL elsewhere, then paste the
            # address the browser lands on (http://localhost:8080/?code=...), even if the page fails to load.
            os.environ.setdefault("OAUTHLIB_INSECURE_TRANSPORT", "1")
            flow.redirect_uri = "http://localhost:8080/"
            url, _ = flow.authorization_url(access_type="offline", prompt="consent")
            print("Open this URL, sign in with the channel's Google account, then paste the final address:\n" + ur
l)
            flow.fetch_token(authorization_response=input("> ").strip())
            creds = flow.credentials
        else:
            creds = flow.run_local_server(port=0, access_type="offline", prompt="consent")
    os.makedirs(os.path.dirname(TOKEN), exist_ok=True)
    with open(TOKEN, "w") as f:
        f.write(creds.to_json())
    return creds


def body_from(meta):
    for key in ("title", "description"):
        if not meta.get(key):
            sys.exit(f"meta.json: '{key}' is required")
```

```
    if len(meta["title"]) > 100:
        sys.exit(f"meta.json: title is {len(meta['title'])} characters (max 100)")
    if len(meta["description"].encode("utf-8")) > 5000:
        sys.exit("meta.json: description is over 5000 bytes")
    if any(c in meta["title"] + meta["description"] for c in "<>"):
        sys.exit("meta.json: '<' and '>' are not allowed in title or description")
    tags = meta.get("tags", [])
    if sum(len(t) + 2 for t in tags) > 500:
        sys.exit("meta.json: tags are over 500 characters in total")
    status = {
        "privacyStatus": meta.get("privacyStatus", "private"),
        "selfDeclaredMadeForKids": bool(meta.get("madeForKids", False)),
        "containsSyntheticMedia": bool(meta.get("containsSyntheticMedia", False)),
    }
    if meta.get("publishAt"):
        if status["privacyStatus"] != "private":
            sys.exit("meta.json: publishAt needs privacyStatus 'private'")
        status["publishAt"] = meta["publishAt"]
    snippet = {
        "title": meta["title"],
        "description": meta["description"],
        "tags": tags,
        "categoryId": str(meta.get("categoryId", "20")),
        "defaultLanguage": meta.get("defaultLanguage", "en"),
        "defaultAudioLanguage": meta.get("defaultAudioLanguage", "en"),
    }
    return {"snippet": snippet, "status": status}


def upload(video, meta_path, dry_run=False):
    with open(meta_path, encoding="utf-8") as f:
        meta = json.load(f)
    body = body_from(meta)
    if not os.path.exists(video):
        sys.exit(f"video not found: {video}")
    if dry_run:
        print(json.dumps(body, ensure_ascii=False, indent=2))
        print(f"OK (dry run): {video}")
        return
    from googleapiclient.discovery import build
    from googleapiclient.errors import HttpError
    from googleapiclient.http import MediaFileUpload

    yt = build("youtube", "v3", credentials=credentials())
    media = MediaFileUpload(video, mimetype="video/mp4", chunksize=8 * 1024 * 1024, resumable=True)
    req = yt.videos().insert(part="snippet,status", body=body, media_body=media, notifySubscribers=True)
    response, tries = None, 0
    while response is None:
        try:
            progress, response = req.next_chunk()
            if progress:
                print(f"uploading {int(progress.progress() * 100)}%")
        except HttpError as e:
            if e.resp.status in (500, 502, 503, 504) and tries < 5:
                tries += 1
                time.sleep(2 ** tries)
                continue
            raise
    vid = response["id"]
    st = response.get("status", {})
    print(f"uploaded: https://youtu.be/{vid}  privacy={st.get('privacyStatus')}  upload={st.get('uploadStatus')}")
    os.makedirs(os.path.dirname(LOG), exist_ok=True)
    with open(LOG, "a", encoding="utf-8") as f:
        f.write(json.dumps({"time": time.strftime("%Y-%m-%dT%H:%M:%S%z"), "video": os.path.basename(video),
                            "id": vid, "title": meta["title"], "privacy": st.get("privacyStatus")},
                           ensure_ascii=False) + "\n")


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("video", nargs="?")
    ap.add_argument("meta", nargs="?")
    ap.add_argument("--dry-run", action="store_true")
    ap.add_argument("--auth", action="store_true", help="sign in and save the token, then exit")
    ap.add_argument("--no-browser", action="store_true")
    a = ap.parse_args()
    if a.auth:
        credentials(no_browser=a.no_browser, force=True)
        print(f"saved token: {TOKEN}")
    elif a.video and a.meta:
        upload(a.video, a.meta, a.dry_run)
```

```
    else:
        ap.print_help()
```